大模型协作记录

提交版附属材料 • 2026-10-29

目录

0 这份记录的来源与边界 2
1 协作分工 2
2 纠错轨迹（全部有产物可核） 2
3 有效的做法（可复用） 4
4 模型在本项目中明确做错的地方（保留） 5
5 对「大模型协作」这项评分的自我评估 5

本文件记录本项目在硬件与提交工程部分使用大模型（Claude）的实际过程：做了什么、模型错在哪、怎么被证据纠正、哪些做法值得复用。

0 这份记录的来源与边界

写作方式：从仓库自身的产物回溯整理——审计脚本与 CSV、修复报告（`_path_audit_20261029/FIX_REPORT.md`）、各阶段的`_*.md` 记录、memory 笔记、以及最后 6 天冲刺会话的逐条对话。

不包含：早期的原始 prompt 全文逐字稿（未留存）。因此下文引用的是**需求原文**与**可验证的结果**，不是逐字对话回放。凡是无法从产物核对的细节，一律不写。

不包含（按赛制要求）：学校、指导教师、个人信息。

---

1 协作分工

角色	负责
人	定义契约：算术格式、寄存器映射、验收判据、红线措辞；决定取舍（例如「6 天只做交付，不接传感器」）；在两个数字矛盾时拍板
模型	实现、审查、批量修复、把散落的数字收进表格、**在被证据反驳时改口**
共同	失败分析。这一节人比模型更重要，因为模型倾向于把「没做到」写成「未来工作」

一个贯穿全程的约束：**模型每给出一个数字，必须同时给出它来自哪个文件。**这条约束直接改变了产出质量——在第 2.2 节的审计案例里，它把「4,552 处硬编码」变成了「16 个文件 126 处」。

---

2 纠错轨迹（全部有产物可核）

2.1 审计口径膨胀：4,552 → 257 → 126 → 0

背景需求：迁移盘符（F: → G:）后，仓库脚本大面积失效，需要知道要改多少。

模型的第一版：全局 grep 绝对路径，报出 **4,552 处**。

为什么错：审计脚本把自己的输出目录（`path_audit.json` 里记录了所有路径）又当成输入扫了一遍，人数虚增；同时把注释、文档、以及 Vivado 自动生成的`build*` 树全部计入。

纠正过程：

1. 加`EXCLUDE_DIR_NAMES` 排除自身输出→ 4,552 降到合理量级；
2. 引入严重度分级：P0（可执行 + 工程路径 + 非注释）/ P1 / P2（注释）/ P3（文档）；
3. 区分「提交必需」与「一次性探针」——后者不进提交包，不需要修。

结果：提交必需的 16 个文件、126 处 P0 **→修完为 0**；全仓剩余 131 处 P0 全部落在一次性探针/watchdog/DRC 脚本里，正确地排除在提交包之外。三档仿真重跑全部 ALL PASS，`work2/out` 60 个文件 md5 与修改前完全一致。

教训：审计的第一件事是定义**什么不算**。口径不清的审计，数字越大越没用。

2.2 UM 参数：模型给了一个错误的建议

问题：attn_axi.v 的参数UM 缺省是 8，但 M=4 的已发布位流是用 12 构建的。

模型的建议：「在块设计里把这个参数覆盖成 12」——**这是错的**。Vivado 不从块设计覆盖 Verilog 参数的缺省值；缺省值就是进位流的值。

怎么被发现的：位流备份attn_axi.v.bak_um12_20261028 与当前树的缺省值不一致，加上位流 md5 只能对应一次构建。冲突无法用「大概是缓存」解释。

最终处理（三道防线）：

1. 把参数改成宏`ATTNAXI_UM，缺省 8，M=4 构建必须定义 12；
2. BITSTREAM_MANIFEST.md 记录每份位流的UM 值与陷阱说明；
3. 逐位测试台作为最终防线——它不关心你怎么配错的，只报MISMATCH。

教训：这个坑的危险性在于**失败是静默的**——综合通过、时序收敛、正常启动、算错结果。只有逐位比对能抓。

2.3 xvlog -d 传字符串：模型连续错三次

目标：给$readmemh 的目录宏MEM_DIR 传入一个路径。

尝试：

```
xvlog.bat -d MEM_DIR="\"../vectors/m4\"" ...   # ERROR: Can not find file: "\"../vectors/m4\""
xvlog.bat -d ATTNAXI_UM=12 ...                 # ERROR: Can not find file: 12
```

模型的行为：坚持这是引号转义问题，连续三次改写引号/反斜杠组合，全部失败。

正确的做法：不传。生成一行头文件，编译在最前面，让`ifndef 跳过缺省：

```
printf ``define MEM_DIR "../vectors/m4"\n' > memdir.vh
xvlog.bat memdir.vh src/rtl/attn_core.v sim/tb/tb_attn_m4.v
```

一次通过，且可复现（sim/run/run_one.sh 就是这么做的）。

教训：工具的失败模式要用**最小复现去否证**，不要靠推理修引号。连错两次同一个方向，就应该换机制而不是换参数。

2.4 被自己写的patching 覆盖

export.py 里的_CACHE 与_BOARD 是新加的定义，却被文件后面原有的赋值语句覆盖了。报错为ModuleNotFoundError，而错误信息指向的地方跟真正的原因相差很远。

处理：删掉后面那两行旧赋值，并在文件头加注释说明这两个变量在**哪里**定义。

教训：修补式编辑必须检查「后面有没有同名的旧赋值」。模型的默认编辑习惯是插入而不是替换，这在脚本类文件里尤其危险。

2.5 统计口径的两次修正（与模型协作的正面案例）

问题	模型做了什么
AU 主效应的旧系数公式是错的 两套 CI 会打架	复核时发现并修正；旧 CI 以 4 个任务为簇聚类，而实际有 80 个独立折 新增 64 名测试被试的配对重采样，得到 [+1.47, +3.12]；并在_数字总表.md 标注「该口径与折均值不同，不能混用」

关键点：模型不是被要求「找一个显著的结果」，而是被要求「把这个数字算对，并且说清楚它是哪个估计量」。

2.6 .gitignore 差点删掉权重

初版写了*.npz，会连src/model/assets/*.npz（量化权重，提交包必需）一起忽略。在git add 前用git status 核对时发现，加了!src/model/assets/*.npz。

教训：通配符否定规则要在**真正的 add 之前**验证，不能等到打包时。

---

3 有效的做法（可复用）

3.1 给模型的三条规定

这三条是全程最有效的：

1. **「这个数字来自哪个文件？」**——不给来源的数字不算数。直接结果：metrics/*.csv 每行都有source 列，metrics.csv 有evidence 列。
2. **「哪些地方你是在推断，而不是在测量？」**——强制区分「量出来的」与「推出来的」。直接结果：README 的「已确立 / 未确立」两张表；本报告第 7 节。
3. **「如果这条被证伪，它会以什么形式出现？」**——让模型的论断可被否证。直接结果：逐位测试台、反向证据（ts_e4 默认实现失败、S 臂种子变号）都留在表里。

3.2 让模型做「减法」

本项目最有价值的模型产出不是新增代码，而是**缩小工作范围**：

- 硬编码审计：从「全仓 4,552 处」缩到「16 个文件 126 处」；
- 构建链调查：发现build_bd_all*.tcl、export.py、fixpt.py、90 个_pl/deploy/*.py **本来就是路径无关的**，真正的修复面比原始计数小一个数量级；
- 6 天冲刺规划：把「四路传感器接入 / 外部同步误差 / 功耗实测」三项从本周目标里划掉，写进限制。**这三项如果硬做，会挤掉全部交付物，而且做不出来。**

3.3 反模式（本项目里真实出现过的）

反模式	后果	对策
用推理修工具报错，不写最小复现	同一个`xvlog` 引号问题错了三次	先写最小复现
数字只给结论不给口径	20.79 / 20.89 / 21.650 被当成矛盾	每个数带口径列
修补式插入不检查同名旧赋值	`export.py` 变量被覆盖	编辑后全文搜变量名
把「没做到」写成「未来工作」	评分项会被当成已完成	单列失败分析节
一次性探针与交付脚本混在一起统计	审计数字失去可操作性	先分必需/非必需

---

4 模型在本项目中明确做错的地方（保留）

不写这一节，这份记录就没有价值。

1. **给了错误的`UM` 覆盖建议**（2.2），且语气是确定的。
2. **在`xvlog -d` 问题上连续三次坚持错误方向**（2.3）。
3. **审计口径一开始没有定义边界**，产出 4,552 这个不可用的数字（2.1）。
4. **修补`export.py` 时没有检查后文同名赋值**（2.4）。
5. **`.gitignore` 通配符险些排除必需资产**（2.6）。
6. 在数字冲突（20.79 / 20.89 / 21.650）时，模型一开始倾向「取一个最可信的」；正确做法是**三个都保留、各自标注口径**。这一条是人纠正模型。

---

5 对「大模型协作」这项评分的自我评估

做得到的：可核对、可复现、有反例、有纠错轨迹、有可移植的 skill 包。每一条结论都能追到一个文件。

做不到的：没有原始 prompt 逐字稿，所以本文是产物回溯而不是对话回放；没有做「绕过模型」的对照基线（同一个人从零手写同样的 RTL 需要多久），所以**无法量化协作带来的效率增益**。这一条是本项评估里的已知缺口。